



第4章： 从头实现GPT模型

GPT 架构内部



LYCHEE

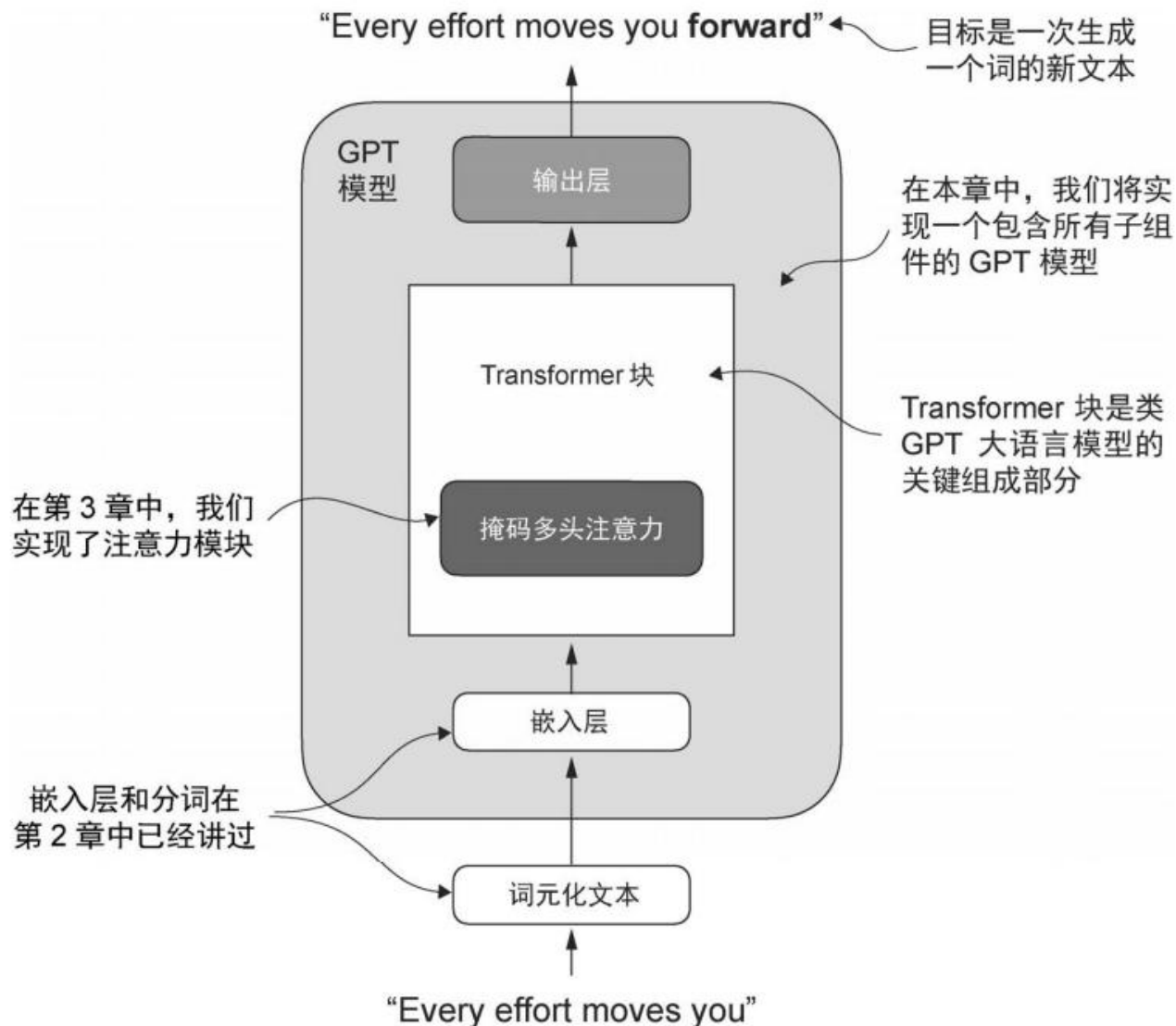


哈尔滨工业大学(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

GPT (Generative Pre-trained Transformer) 是一种以“**预测下一个 token**”为核心目标的深度神经网络。

它的整体结构可以理解为：把许多个完全相同的“Transformer Block”按垂直方向一层层堆叠起来，形成一个深层网络。

以“GPT-2 Small”为例，模型大约包含 1.24 亿个**参数**，即**模型内部可学习的权重**，它们是一组可以被训练调整的数字，用来把输入表示逐步变换，最终输出对“下一个 token”的概率预测。



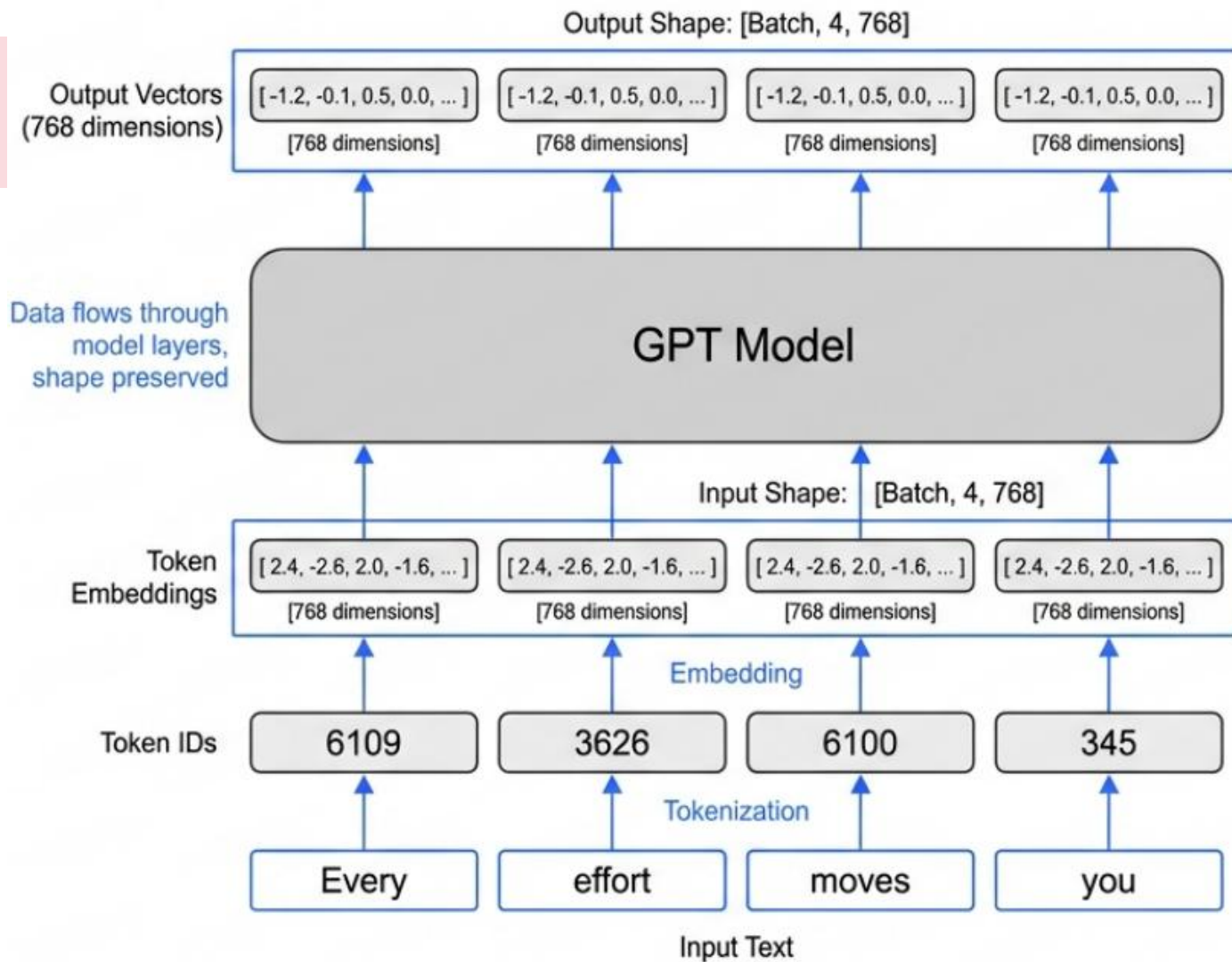
从文本到高维向量 From Text to High-Dimensional Vectors

在进入模型计算之前，人类语言必须先被转换成机器能够处理的“几何表示”，也就是向量。

(1) 分词 (Tokenization)：把文本切分成一个个 token，并用词表把每个 token 映射成整数 ID。

(2) 嵌入 (Embedding)：把这些整数 ID 查表转换成稠密向量，让离散的符号变成可计算的连续表示。

(3) 维度保持 (Dimension Preservation)：以 GPT-2 Small 为例，每个 token 会对应一个 768 维的向量，因此整段输入在模型中的张量形状通常是 [Batch, Sequence, 768]，并且这个 768 维的特征维度会在后续各层中始终保持不变，便于层与层之间稳定地传递信息。

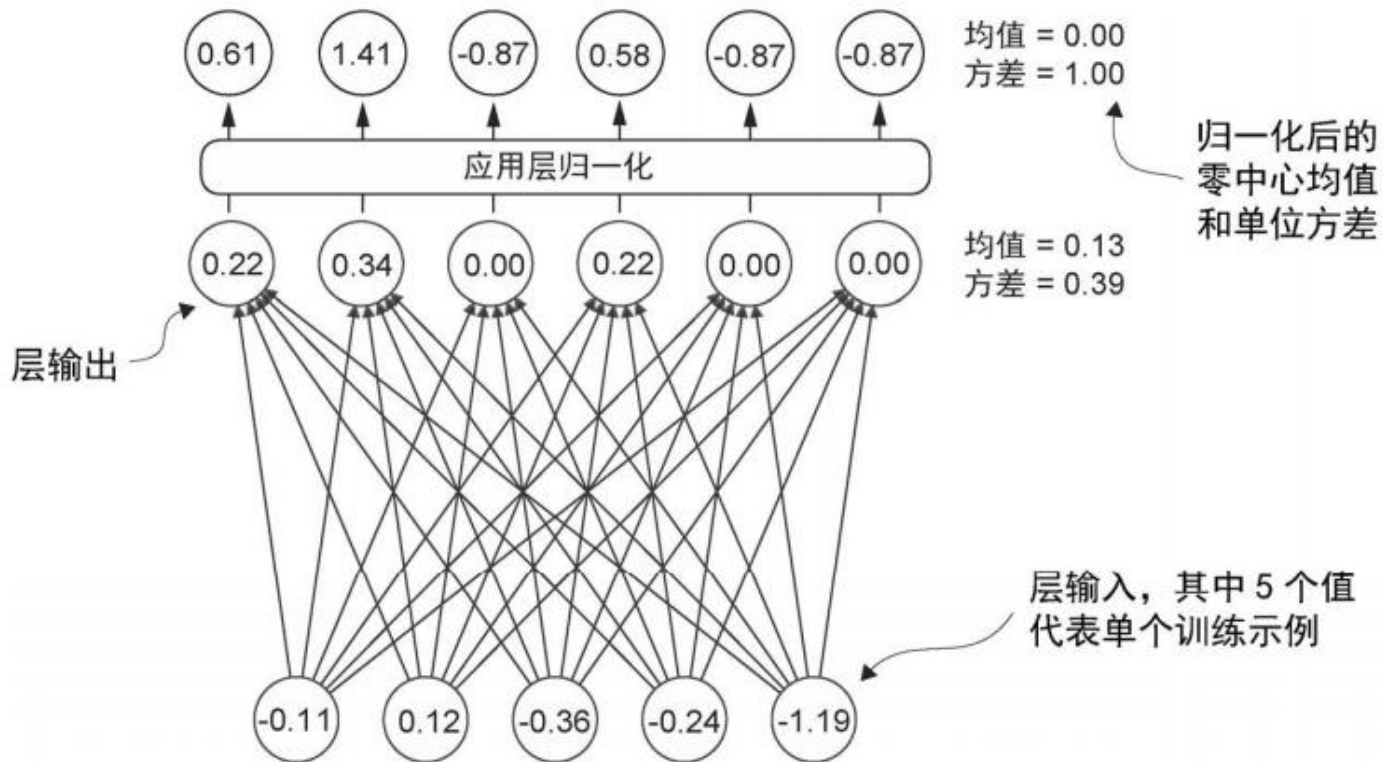


数值稳定性：层归一化 Layer Normalization

在深度网络里，数据会一层一层往下传。随着层数变深，激活值的分布可能会不断漂移，变得忽大忽小，出现**梯度爆炸或梯度消失**等不稳定现象。

层归一化 (Layer Normalization, LayerNorm) 就像一个“稳定器”。它会在每一层的输出上做一次标准化处理：把数据重新居中并缩放，让这一层的输出分布更规整，通常会被调整到**均值接近 0、方差接近 1**的范围。

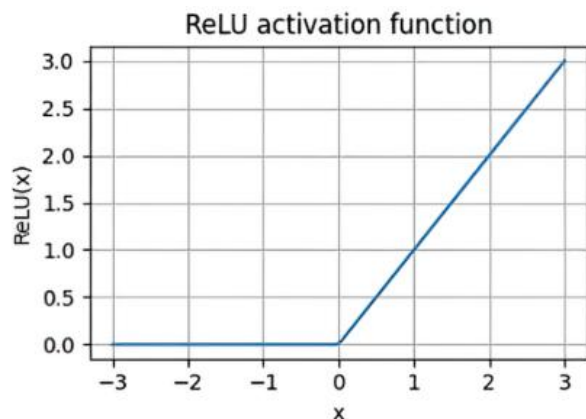
这样做的好处是把内部信号维持在一个更容易训练的区间，让优化过程更稳定，减少发散风险，并帮助模型更快、更可靠地收敛。



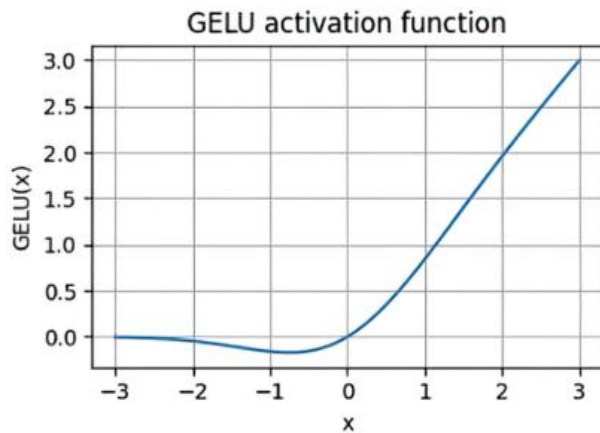
激活函数：从 ReLU 到 GELU

核心原理

激活函数决定了神经元在接收到输入后要“输出多少”，也可以理解为决定神经元是否被激活、以及激活的强度。



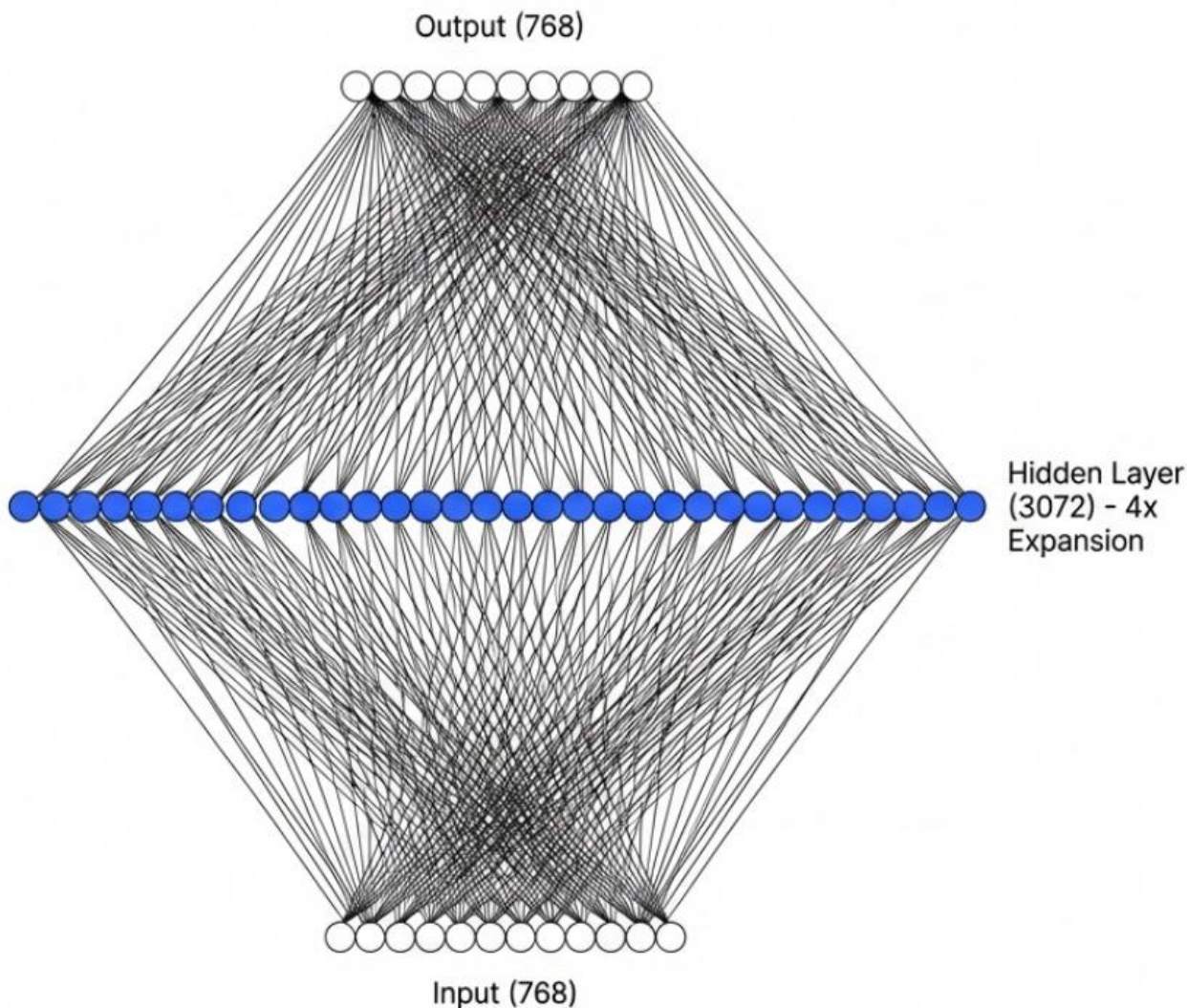
ReLU (Rectified Linear Unit) 是传统且常用的选择。它在 0 处有一个“硬阈值”：当输入小于 0 时，输出会被直接截断为 0；当输入大于 0 时，输出等于输入本身。这样的好处是简单、高效，但缺点是负半轴完全没有输出，某些神经元可能长期落在负区间，梯度变成 0，出现“死 ReLU”的问题。



GELU (Gaussian Error Linear Unit) 是大模型里更常见的标准做法。它的行为整体类似 ReLU，但不是硬切割，而是更平滑、带有“概率意义”的曲线过渡。

在接近 0 的小负值区域并不会完全归零，而是有一个轻微的下凹与连续变化，这使得负区间仍然可以产生非零梯度，让优化过程更细腻、更稳定。

前馈网络 FFN



注意力层之后通常会接一个**前馈网络 (FeedForward Network, FFN)**。它的特点是“逐 token 独立处理”，也就是说每个位置的向量只在自己的特征维度上做变换，不和其他位置直接交互，因此可以理解作为一种局部的非线性加工模块。

(1) 扩张 (Expansion)：把每个 token 的 768 维输入投影到更高维的 3072 维空间（约 4 倍）。

(2) 处理 (Processing)：在高维空间里通过线性变换与 GELU 激活，让更复杂的特征被分离出来并形成非线性表达。

(3) 收缩 (Contraction)：再把 3072 维投影回 768 维，恢复到主干通道的维度，方便继续传给下一层。

捷径连接 Residual Connection



LYCHEE



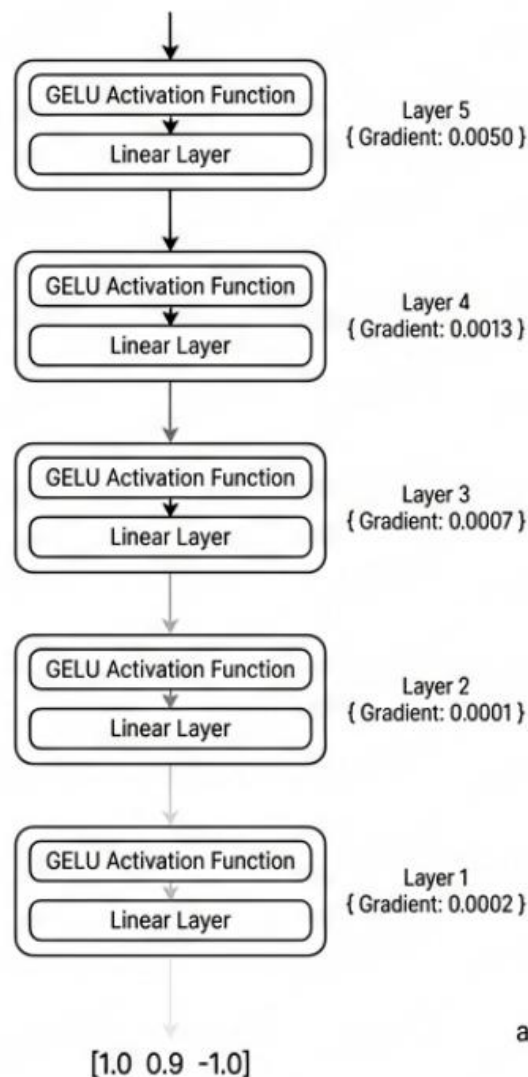
哈尔滨工业大学(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

深层网络常见的困难之一是“**梯度消失**” (**Vanishing Gradient**)：反向传播时，学习信号在层层传递过程中逐渐变弱，越靠前的层越难得到有效更新，训练就会变慢甚至不稳定。

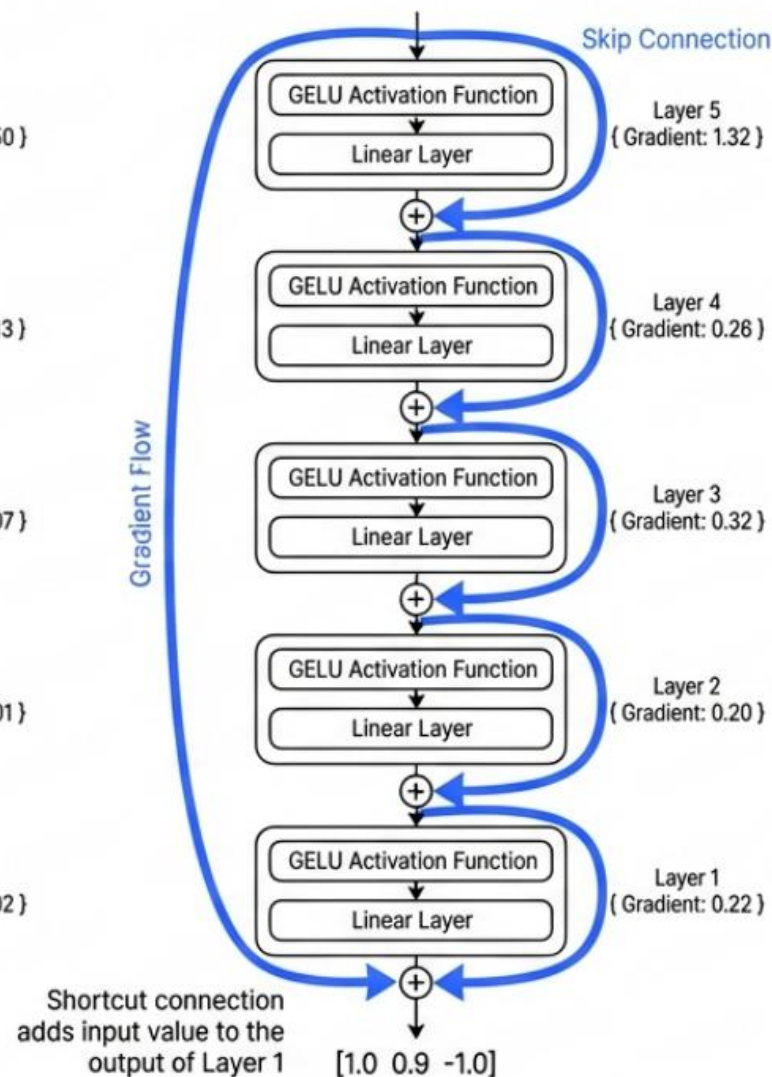
捷径连接 (Shortcut Connection)，也叫**残差连接 (Residual Connection)**，用一种很直接的方式缓解这个问题：把某一层的输入直接加到该层的输出上，也就是 $x + \text{Layer}(x)$ 。这样做相当于给信息和梯度提供了一条“直达通道”，让它们可以绕过复杂的非线性变换块，更顺畅地向前、向后传播。

即使是网络很深，最前面的层也能收到足够强的更新信号，从而更稳定、更容易训练。

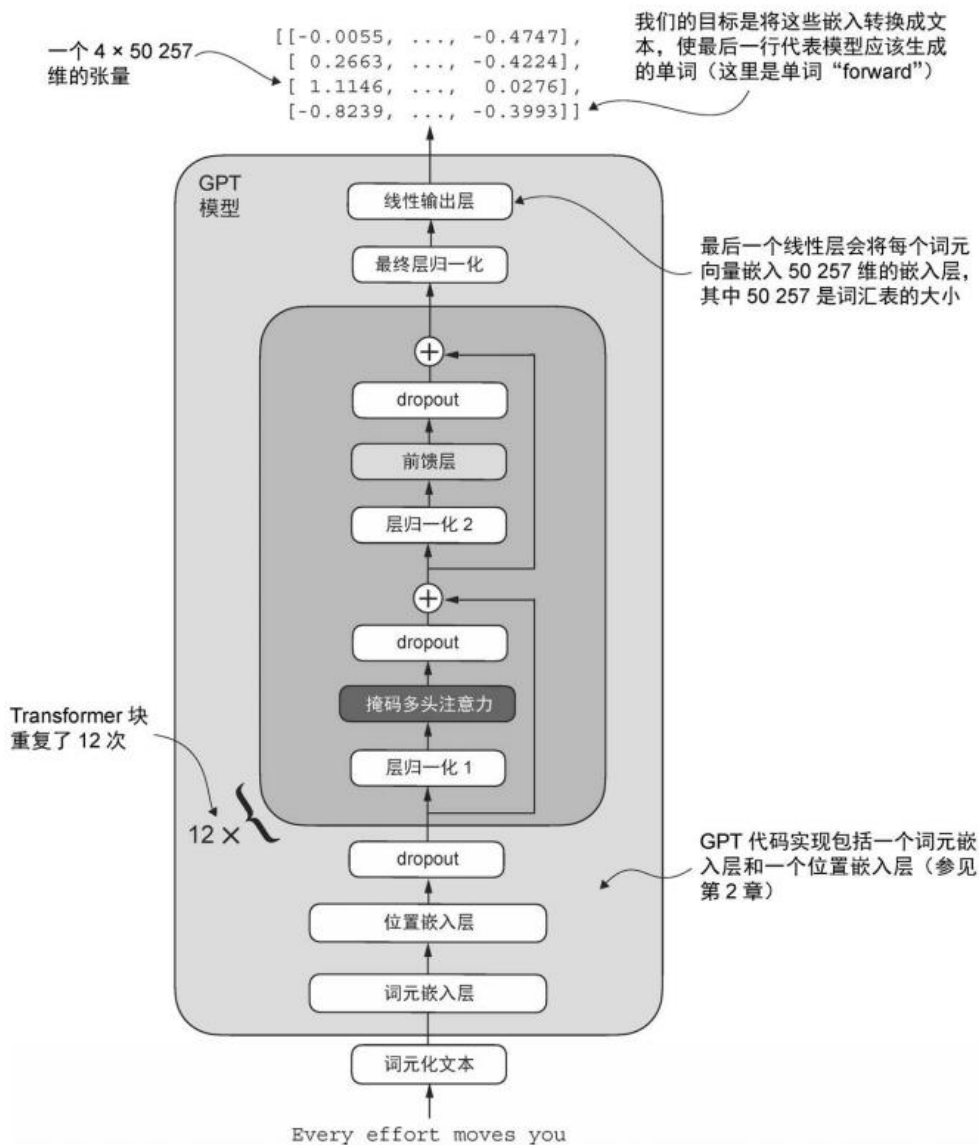
Standard Deep Network



Residual Network



组装 Transformer Block



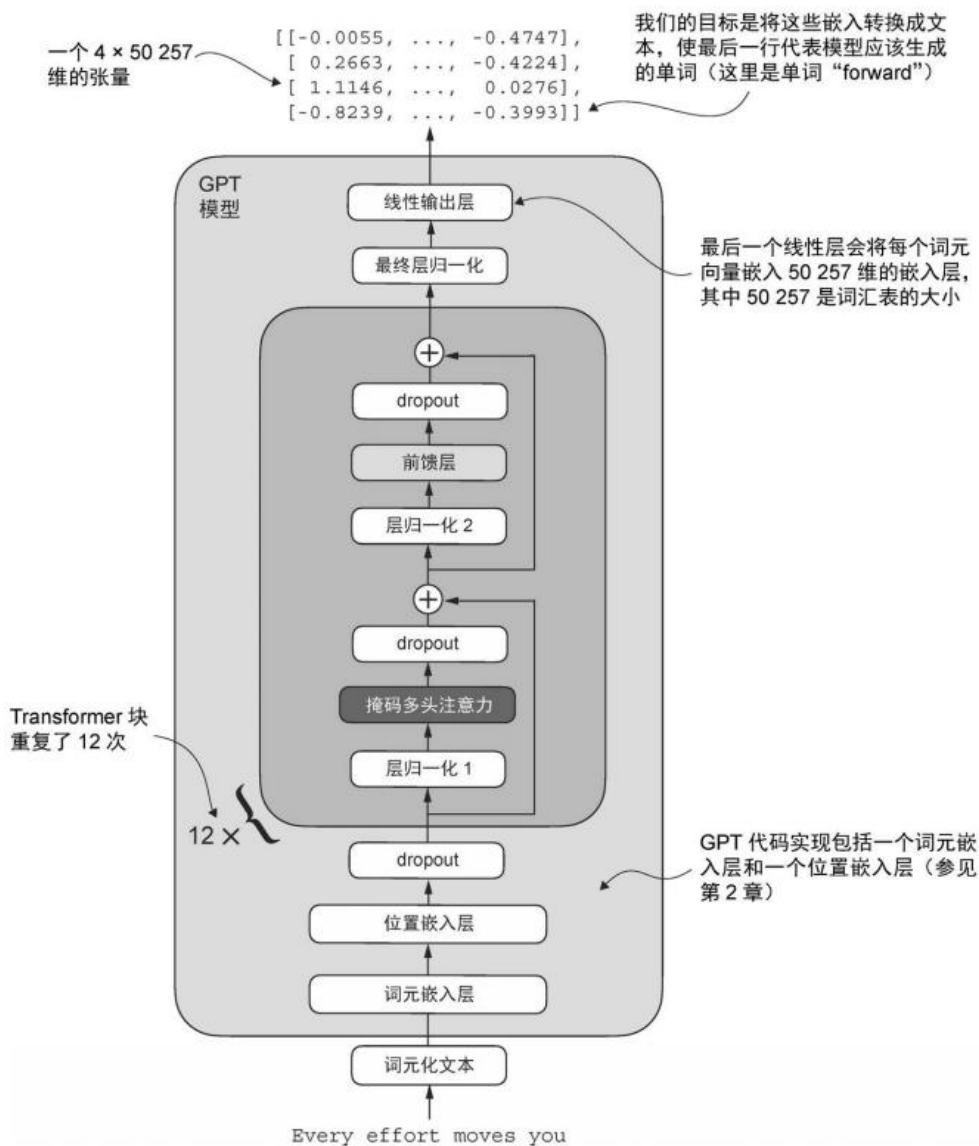
现在我们把前面介绍的组件（LayerNorm、Attention、FFN、残差连接）组合起来，形成 GPT 中最核心、反复堆叠的基本单元：**Transformer Block。**

注意力路径（Attention Path），负责让不同位置的 token 彼此“交流”，根据上下文建立关联。

处理路径（Processing Path），也就是前馈网络，让每个 token 在自己的特征空间里“独立思考”，进一步提炼和变换信息。

Transformer Block 的输入与输出向量形状保持一致。在 GPT-2 Small 中，这个维度是 768，因此输入是 768 维，经过注意力与 FFN 后输出仍然是 768 维。

完整的 GPT-2 Small 架构



GPT-2 Small 的整体结构可以分为三部分：**输入端（Base）、主干（Body）和输出端（Head）。**

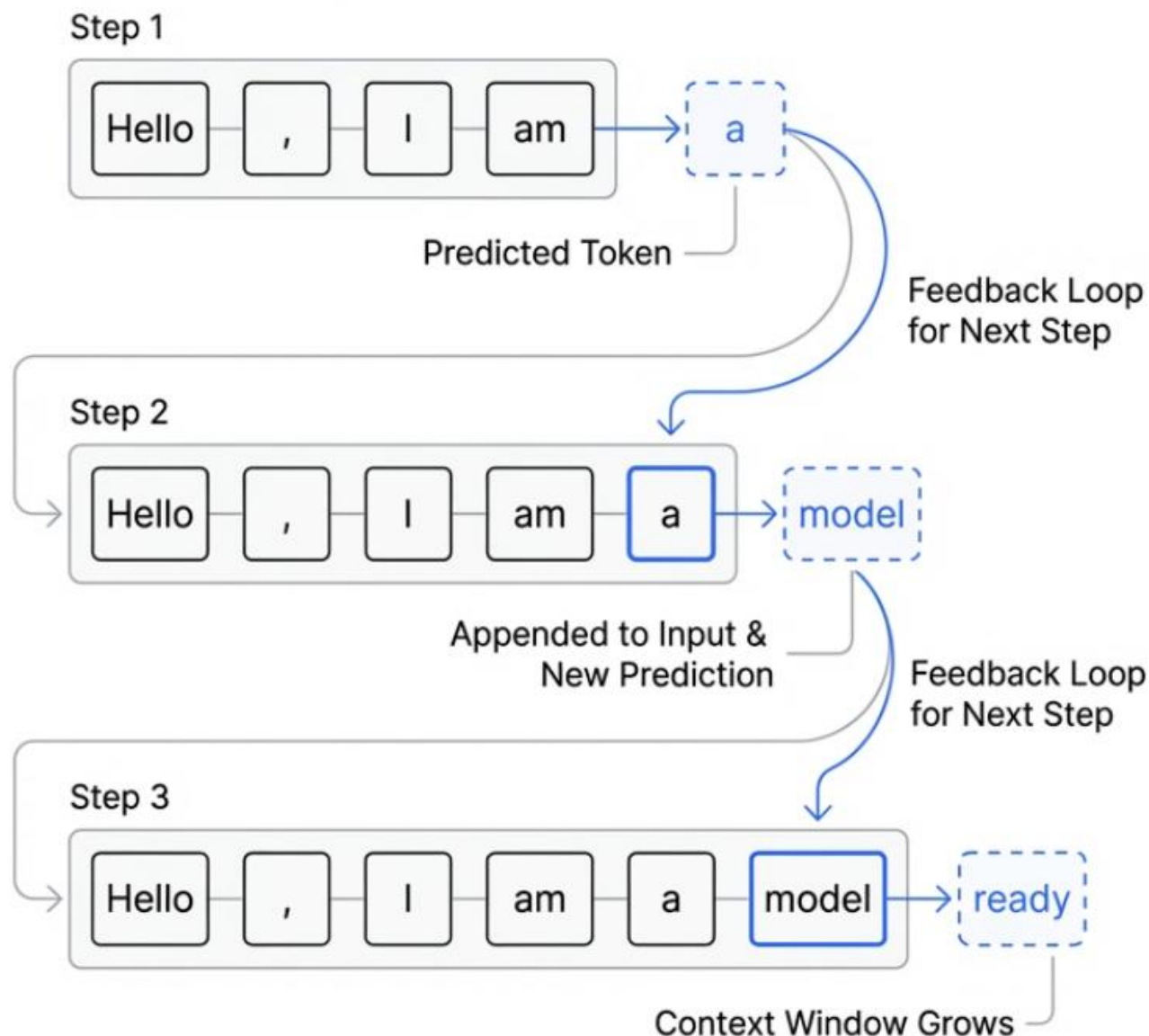
- 在**输入端**，模型接收分词后的 token，并将 token embedding 与位置 embedding 相加，形成初始的输入表示。
- 进入**主干**后，数据会依次通过 12 个完全相同的 Transformer Block；随着层数加深，表示会逐步变得更抽象、更具上下文信息，从而更好地捕捉语义与长距离依赖。
- 最后在**输出端**，模型先经过归一化层，再通过线性投影把 768 维的向量映射到整个词表大小的空间，最终得到一个 50,257 维的概率分布，用来表示“下一个 token”在词表中各词的预测概率。

文本生成：滑动窗口

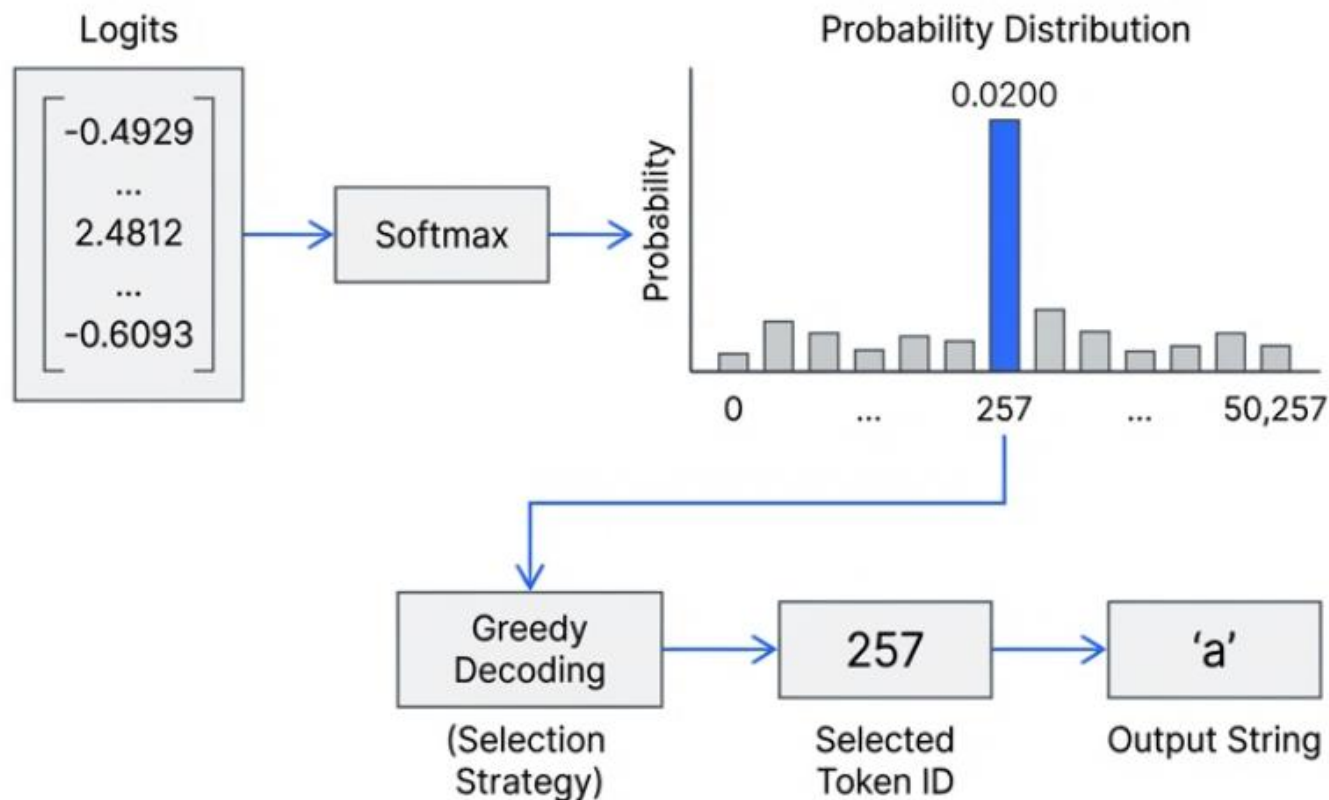
模型的网络结构是固定的，但生成过程是动态的。

GPT 通过**自回归 (autoregressive)** 循环来写文本：
先给定已有上下文作为输入，例如“Hello, I am”；模型根据这段上下文预测下一个 token，比如预测为“a”；然后把预测到的“a”追加回输入序列，形成新的上下文“Hello, I am a”；接着用这段更长的序列再次输入模型，继续预测下一个 token。

每完成一次循环，上下文窗口就会新增一个 token，模型也就一步一步把文本续写出来。



解码：从输出向量到文字 Decoding



把“数学输出”变成真正的文字，发生在模型的**解码头 (decoding head)** 里。

模型会为词表中的每个词输出一个**原始分数 (logits)**，例如对 50,257 个词分别给出打分。接着通过 Softmax，把这些分数归一化成概率分布，表示每个词作为“下一个 token”的可能性大小。

然后使用一种选择策略来决定最终输出，比如**贪心解码 (Greedy Decoding)** 会直接选取概率最高的那个 token。

最后把选中的 token ID（例如 257）查词表还原成字符串，得到对应的文字（例如 “a”）。